Interview Guide - Prompt Evaluation Studio Pro

Interview Guide — Prompt Evaluation Studio Pro

30-Second Explanation

"I built Prompt Evaluation Studio Pro — a Streamlit platform for prompt engineering using Google Gemini. It lets users create, compare, evaluate, and optimize AI prompts with version control, analytics, and automated scoring across nine quality criteria."

2-Minute Explanation

"I developed a full-stack Python application called Prompt Evaluation Studio Pro for professional prompt engineering.

The platform uses Streamlit for the frontend, SQLite with SQLAlchemy for persistence, and the Google Gemini API via the google-genai SDK.

Key features include a Prompt Playground with streaming and JSON mode, side-by-side comparison of up to five prompt variants, an AI Evaluation Engine that scores outputs on nine criteria out of 100, and a Prompt Optimizer that generates multiple improved versions.

I implemented proper architecture with separate service layers, Pydantic validation, bcrypt authentication, version control with diff viewing, and export to PDF, CSV, and JSON.

The project demonstrates real prompt engineering techniques — role prompting, structured JSON output, prompt chaining, and systematic evaluation — not just API calls."

5-Minute Explanation

[Include 2-minute explanation plus:]

"Architecturally, I used a layered design. The Streamlit frontend handles UI only. A ServiceContainer in the backend wires up AuthService, PromptService, GeminiService, EvaluationService, and others. Each service has a single responsibility.

For prompt evaluation, I designed structured system prompts that instruct Gemini to return JSON with nine scoring criteria. The EvaluationService parses and validates this, stores results in the evaluations table, and surfaces explanations plus optimized prompt suggestions.

The comparison module runs identical inputs through multiple prompts, measures response time and token usage, uses Gemini for quality scoring, and applies weighted scoring to declare a winner.

I wrote 23 pytest tests covering unit, integration, database, and API layers with mocked Gemini calls so tests run without an API key.

For DevOps, I added Docker, docker-compose, and documented Streamlit Cloud deployment. The project includes complete documentation — architecture, database schema, API reference, and this interview guide."

10-Minute Explanation

[Include 5-minute explanation plus:]

"Let me walk through the user workflow and technical decisions.

When a user signs up, bcrypt hashes their password and five built-in prompt templates are seeded. In the Playground, they configure system prompt, user prompt, temperature, top-p, top-k, max tokens, and safety settings. Input variables use {{name}} syntax and are substituted before the API call.

Every run auto-saves as an experiment and logs an analytics event. Users can then send the output to the Evaluation Engine, which uses role prompting and structured JSON to score accuracy, completeness, hallucination risk, grammar, structure, professionalism, formatting, readability, and prompt effectiveness.

The Optimizer uses few-shot patterns and constraint specification to generate three distinct improved versions with explained improvements.

Version control works like Git for prompts — create snapshots, view diffs, restore previous versions, tag and categorize.

The analytics dashboard uses Plotly for daily usage charts, category pie charts, and response time trends.

I handled errors comprehensively — invalid API keys, rate limits, timeouts, empty prompts, JSON parse failures, and database errors all have custom exception classes with user-friendly messages.

This project taught me that prompt engineering is an engineering discipline — prompts need versioning, testing, evaluation metrics, and iteration loops, just like software."

Architecture Explanation

"I chose Streamlit because it enables rapid development of data-focused UIs with minimal frontend code. The trade-off is less UI customization, but for a prompt engineering tool, the priority is functionality.

The service layer pattern keeps business logic testable. Streamlit pages never touch the database directly — they always go through get_services() context manager which handles session lifecycle and commits.

Gemini integration is isolated in GeminiService with methods for generate, stream, and JSON mode. This means if we switch to another LLM, only one file changes."

Database Explanation

"Six main tables: users, prompts, prompt_versions, experiments, evaluations, and analytics, plus a settings key-value store.

Prompts have a one-to-many relationship with versions for version control. Experiments link optionally to prompts. Evaluations link optionally to experiments.

I track usage_count and success_rate on prompts for analytics. Analytics events capture every significant action for dashboard charts."

Prompt Engineering Explanation

"I applied role prompting in evaluation and optimization system prompts. Structured JSON output ensures parseable scores. The evaluation prompt inverts hallucination_risk when computing overall_score.

Templates use variable substitution for reusability. The comparison module chains generation then scoring. Error recovery retries JSON parsing if the first attempt fails."

Gemini Integration

"I use the google-genai SDK with the Client class. Safety settings map to HarmBlockThreshold enums. JSON mode uses response_mime_type application/json.

Streaming uses generate_content_stream for real-time Playground output. Connection testing sends a minimal prompt to verify the API key."

Expected HR Questions

Q: Why did you build this project?
A: To demonstrate practical prompt engineering skills and full-stack Python ability for AI roles.

Q: How long did it take?
A: Approximately 2-3 weeks including documentation and testing.

Q: What was the hardest part?
A: Designing the evaluation scoring system to be consistent and designing weighted comparison metrics.

Q: Would you use this in production?
A: With PostgreSQL instead of SQLite, proper secret management, and rate limiting — yes, the architecture supports it.

Q: What did you learn?
A: Prompt engineering requires systematic evaluation, version control, and iteration — not just clever wording.

Expected Technical Questions

Q: Why Streamlit over Flask/FastAPI + React?
A: Faster development for data/AI apps; sufficient for portfolio demonstration.

Q: How do you handle API rate limits?
A: Custom RateLimitError exception with user-friendly message; retry logic can be added at service layer.

Q: How is password security handled?
A: bcrypt with unique salts per password via bcrypt.gensalt().

Q: How do you test without Gemini API?
A: unittest.mock patches on genai.Client in pytest.

Q: Explain your service container pattern.
A: Lazy-loaded services sharing one SQLAlchemy session per request context.

---

50 Interview Questions with Answers

General (1-10)

1. What is Prompt Evaluation Studio Pro?
A platform for creating, testing, evaluating, and optimizing AI prompts using Google Gemini.

2. What problem does it solve?
Prompt engineering lacks tooling for systematic testing, comparison, and quality measurement.

3. Who is the target user?
Developers, prompt engineers, and AI practitioners who need structured prompt workflows.

4. What makes it different from ChatGPT?
It provides version control, multi-prompt comparison, automated scoring, and experiment tracking.

5. What tech stack did you use?
Python, Streamlit, Gemini API, SQLite, SQLAlchemy, Pydantic, Plotly.

6. Is it open source?
Yes, MIT licensed.

7. Can it work without internet?
UI works offline; Gemini features require internet.

8. How many features does it have?
12 major modules covering the full prompt lifecycle.

9. How do you measure prompt quality?
Nine criteria scored 0-100 by the AI Evaluation Engine.

10. What is the overall architecture pattern?
Layered architecture with service-oriented design.

Python & Backend (11-20)

11. Why Python 3.12?
Modern type hints, performance improvements, industry standard for AI.

12. What is Pydantic used for?
Request/response validation with type safety.

13. Explain SQLAlchemy usage.
ORM mapping Python classes to SQLite tables with relationship management.

14. What is the ServiceContainer?
Factory providing lazy-loaded services with shared DB session.

15. How are database sessions managed?
Context manager with auto commit/rollback in get_session().

16. How do you handle errors?

Custom exception hierarchy: AppError, GeminiAPIError, ValidationError, etc.

17. What logging approach do you use?
Python logging to console and logs/app.log file.

18. How are environment variables loaded?
python-dotenv via config/settings.py with pydantic Settings model.

19. What design patterns are used?
Service layer, factory, repository-like pattern, dependency injection.

20. How is code organized?
Modular folders: frontend, backend, services, database, models, utils, tests.

Database (21-25)

21. Why SQLite?
Zero configuration, perfect for portfolio and local development.

22. How many tables?
7: users, prompts, prompt_versions, experiments, evaluations, analytics, settings.

23. How does version control work in DB?
prompt_versions table stores snapshots; prompts.current_version tracks latest.

24. How are analytics stored?
analytics table with event_type, metrics, and JSON metadata.

25. How do you backup data?
Admin panel copies SQLite file to backups/ directory.

Gemini & AI (26-35)

26. Which Gemini models are supported?
gemini-2.0-flash, gemini-2.0-flash-lite, gemini-1.5-flash, gemini-1.5-pro.

27. How is streaming implemented?
generate_content_stream yields text chunks to Streamlit placeholder.

28. What are safety settings?
HarmBlockThreshold levels controlling content filtering.

29. How does JSON mode work?
response_mime_type set to application/json in generation config.

30. How do you reduce hallucinations in prompts?
Optimizer adds constraints; evaluator scores hallucination_risk.

31. What is role prompting?
Assigning an expert persona in the system prompt.

32. What is prompt chaining?
Playground → Evaluation → Optimizer → Playground refinement loop.

33. How does comparison pick a winner?
Weighted score across quality, consistency, readability, and speed.

34. How are tokens estimated?
Word count × 1.3 heuristic in text_metrics.py.

35. How do you validate JSON from Gemini?
json.loads with fallback stripping markdown fences.

Frontend & UX (36-40)

36. Why Streamlit?
Rapid Python-native UI for data and AI applications.

37. How is navigation handled?
Sidebar buttons setting session_state.current_page.

38. How is dark mode implemented?
User theme stored in database; Streamlit dark theme config.

39. How are charts rendered?
Plotly via st.plotly_chart with plotly_dark template.

40. How is auth session managed?
Streamlit session_state with optional remember_token in database.

Testing & DevOps (41-45)

41. How many tests?
23 pytest tests, all passing.

42. What test types?
Unit, integration, database, API mock, startup tests.

43. How is Docker configured?
Python 3.12-slim image, streamlit on port 8501, volume mounts.

44. Can it deploy to Streamlit Cloud?
Yes, with secrets for API key and database URL.

45. How do you run CI?
pytest in GitHub Actions (documented in TESTING.md).

Advanced (46-50)

46. How would you scale this?
PostgreSQL, Redis caching, Celery for async generation, multi-user RBAC.

47. How would you add team collaboration?
Shared prompt libraries, role-based permissions, comment threads on versions.

48. How would you add A/B testing?
Extend comparison module with statistical significance testing.

49. What security improvements for production?
JWT tokens, HTTPS, secrets manager, input sanitization, rate limiting.

50. What's next for the project?
Multi-model support (OpenAI, Claude), automated regression testing, prompt CI/CD pipelines.